

Atlas

ტენდერების ინტელექტუალური პლატფორმა

საქართველოს ტექნიკური უნივერსიტეტი — ჰაკათონი
ტექნიკური დოკუმენტაცია · 2026

B2B პლატფორმა სახელმწიფო შესყიდვების ტენდერებში მონაწილე კომპანიებისთვის. ჰიბრიდული retrieval (dense + sparse), cross-encoder reranker და Gemini RAG — მაღალი მოგების შანსის ტენდერების შერჩევა და ფასის რეკომენდაცია.

დოკუმენტი შეიცავს: მიმოხილვას, არქიტექტურას,
ტექნოლოგიურ სტეკს და pipeline-ის დეტალებს.

მიმოხილვა

ტენდერების ინტელექტუალური პლატფორმა

საქართველოს ტექნიკური უნივერსიტეტი — ჰაკათონი ტექნიკური დოკუმენტაცია · 2026

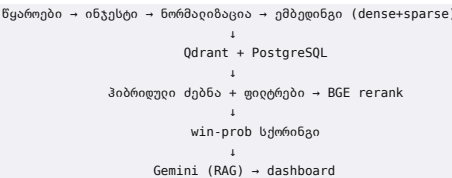
მიმოხილვა

Atlas არის B2B პლატფორმა, რომელიც შექმნილია იმ კომპანიებისთვის, რომლებიც სახელმწიფო შესყიდვების ტენდერებში (ITT/RFP) მონაწილეობენ. სისტემა აანალიზებს გუნდის კომპეტენციების პროფილს, ეძრება მას ტენდერების კორპუსს და გვთავაზობს:

- მაღალი მოგების შანსის მქონე ტენდერებს — კომპეტენციის, ვადების, მოთხოვნებისა და გეოგრაფიის გათვალისწინებით;
- რეკომენდებულ ფასს — ისტორიული გამარჯვებული ფასების მოდელზე დაფუძნებულ შეფასებას;
- ციტირებად ინსაიტს — ყოველი რეკომენდაცია მოყვება წყაროს მითითებით და ახსნა-განმარტებით.

ძებნა ორფაზიანია: ფართო retrieval (ჰიბრიდული dense+sparse) იძლევა recall-ს, შემდეგ cross-encoder reranker ახშობს precision-ს, ბოლოს Gemini (RAG) აწარმოებს ვალიდაციას და ფასის რეკომენდაციას.

არქიტექტურა (ერთი შეხედვით)



პრინციპი: იაფი ფართო ძვირი ზუსტი. ჯერ recall, შემდეგ precision.

ტექნოლოგიური სტეკი (მოკლედ)

ფენა	ტექნოლოგია	რატომ ეს არჩევანი
ინჟესტი	PyMuPDF, Tesseract (OC...	მრავალფორმატიანი ფაილე...
Dense ემბედინგი	`gemini-embedding-001`	მაღალხარისხიანი მრავალ...
Sparse ემბედინგი	BGE-M3 (`BAAI/bge-m3`)	ნასწავლი ლექსიკური წონ...
ვექტორული ბაზა	Qdrant (hybrid search)	native dense+sparse na...
რერანკინგი	`bge-reranker-v2-m3`	cross-encoder — ზუსტი ...
მეტა-მონაცემი	PostgreSQL	კომპეტენციის პროფილები...
გენერაცია	Gemini (`gemini-2.5-pr...	RAG ინსაიტი, eligibili...

Frontend	React / Next.js dashbo...	KPI მონიტორინგი + ქართ...
----------	---------------------------	---------------------------

სტეკის ცენტრში — ჰიბრიდული retrieval: Google-ის dense ემბედინგი იჭერს სემანტიკას, BGE-ის sparse ემბედინგი ზუსტ ტერმინებსა და CPV კოდებს. ორივე ერთად ინდექსდება Qdrant-ში, ირჩევა fusion-ით, შემდეგ იხვეწება BGE რერანკერით და ისახება Gemini-ს RAG ფენაში.

დოკუმენტაციის რუკა

- ARCHITECTURE.md — სრული სისტემური არქიტექტურა, მონაცემთა ნაკადები, კომპონენტების ურთიერთქმედება;
- TECH_STACK.md — ტექნოლოგიური არჩევანის დეტალური დასაბუთება და ალტერნატივები;
- PIPELINE.md — პროცესის საფეხურები (ინჯესტი შენახვა ძებნა რერანკინგი ფასი).

ლიცენზია

ჰაკათონის პროექტი — საქართველოს ტექნიკური უნივერსიტეტი, 2026.

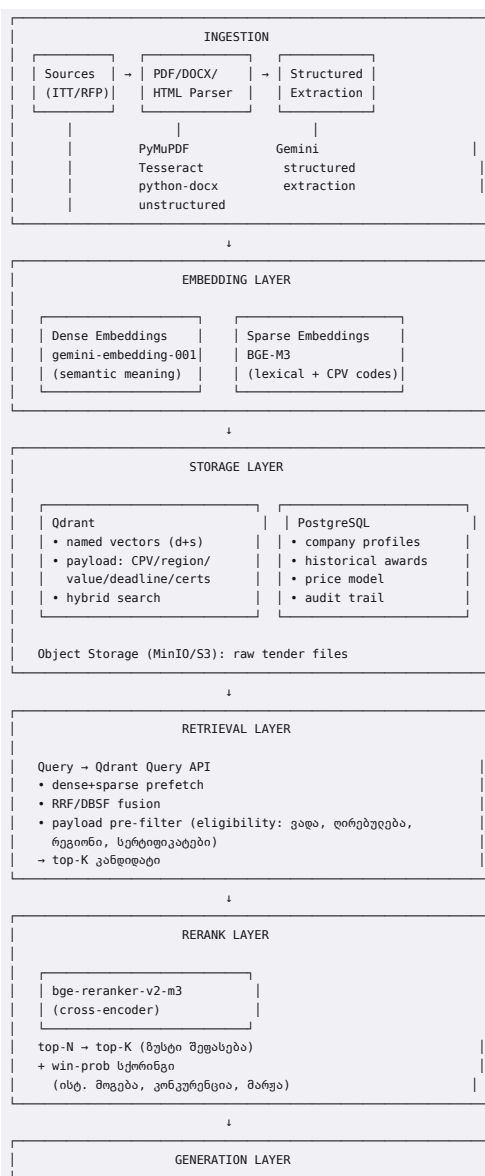
არქიტექტურა

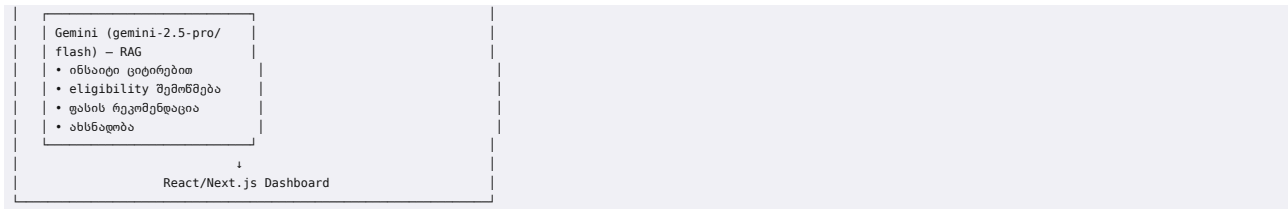
სისტემური მიმოხილვა

Atlas-ის არქიტექტურა დაფუძნებულია ორფაზიანი retrieval პრინციპზე:

- 1.. ფართო retrieval (იაფი, recall-ზე ორიენტირებული) — ჰიბრიდული dense+sparse ძებნა Qdrant-ში;
- 2.. ზუსტი rerank (ძვირი, precision-ზე ორიენტირებული) — cross-encoder BGE-რერანკერი top-N კანდიდატზე;
- 3.. RAG გენერაცია — Gemini აწარმოებს ციტირებად ინსაიტს, eligibility-შემოწმებას და ფასის რეკომენდაციას.

მონაცემთა ნაკადი (Data Pipeline)





ძირითადი კომპონენტები

1. Ingestion Layer

- ფუნქცია: ტენდერის დოკუმენტების (PDF/DOCX/HTML) ერთიანი პარსინგი და ნორმალიზება;
 - OCR: Tesseract სკანირებული დოკუმენტებისთვის;
 - სტრუქტურირებული ამოღება: Gemini structured extraction კანონიკური სქემა Tender (CPV, შემსყიდვედი, ვადა, ღირებულება, მოთხოვნები, მოთხოვნის დონე);
 - Chunking: გრძელი დოკუმენტების სემანტიკური ნაწილებად დაყოფა;
 - Scheduler: ყოველდღიური წუთისობრივი ingestion ახალი ტენდერებისთვის.

2. Embedding Layer

- Dense: gemini-embedding-001 — მრავალენოვანი სემანტიკური მნიშვნელობა (ქართულის ჩათვლით);
- Sparse: BGE-M3 — ნასწავლი ლექსიკური წონები იჭერს ზუსტ ტერმინებსა და CPV კოდებს, რომლებსაც dense კარგავს.

3. Storage Layer

- Qdrant: ჰიბრიდული ვექტორული ბაზა — point = named vectors (dense+sparse) + payload (CPV/ვადა/რეგიონი/ღირებულება). payload ინდექსირებულია ფილტრებისთვის;
- PostgreSQL: კომპეტენციის პროფილები, ისტორიული award-ფასები, audit trail;
- Object Storage (MinIO/S3): ორიგინალი raw დოკუმენტები.

4. Retrieval Layer

- Query API: Qdrant Query API — dense+sparse prefetch + fusion (RRF/DBSF);
- Pre-filter: eligibility ფილტრაცია payload-ზე (ვადა, ღირებულება, რეგიონი, სერტიფიკატები);
 - Output: top-K კანდიდატი rerank-ისთვის.

5. Rerank Layer

- Cross-encoder: bge-reranker-v2-m3 — ზუსტი შეფასება მცირე კანდიდატთა ჯგუფზე (top-N);
- Win-probability scoring: ისტორიული მოგების ალბათობა, კონკურენციის ინტენსივობა, მარჯის პოტენციალი;
 - ახსნადობა: რომელი მოთხოვნა დაემთხვა და რატომ მოხვდა მაღლა.

6. Generation Layer

- LLM: Gemini (gemini-2.5-pro/flash);
- RAG: ინსაიტი წყაროების ციტირებით, eligibility შემოწმება, ფასის რეკომენდაცია;
- Structured output: JSON სქემა dashboard-ისთვის.

7. Frontend

- React / Next.js dashboard;
- ქართულენოვანი შეჯამება — ბუნებრივი, არა ლიტერატურული თარგმნა;
- KPI მონიტორინგი — win-rate, საშუალო მარჟა, პროგნოზირებული შემოსავალი.

დიზაინის პრინციპები

- იაფი ფართო ძვირი ზუსტი: retrieval recall-ზე, rerank precision-ზე;
- ჰიბრიდულობა: dense + sparse ერთად — არცერთი მარტო საკმარისი არ არის CPV-ზე ორიენტირებულ კორპუსში;
- ციტირებადობა: ყველა რეკომენდაცია მოყვება წყაროს ბმული/ID-ით;
- ახსნადობა: მოდელი არ უნდა იყოს "შავი ყუთი" — ნებისმიერ შეფასებას თან ახლავს მიზეზი;
- ადაპტურობა: LLM შეიძლება შეიცვალოს provider-ზე დამოკიდებულების გარეშე (OpenAI, Claude და ა.შ.).

ტექნოლოგიური სტეკი

ცენტრალური იდეა

Atlas-ის სტეკის გული არის ჰიბრიდული retrieval: Google-ის dense ემბედიנגი იჭერს სემანტიკურ მნიშვნელობას (მათ შორის ქართულ ენაზე), BGE-ის sparse ემბედიנגი კი — ზუსტ ტერმინებსა და CPV კოდებს. ორივე ერთად ინდექსდება Qdrant-ში, ირჩევა fusion-ით, შემდეგ იხვეწება BGE რერანკერით და საბოლოოდ მუშავდება Gemini-ს RAG ფენაში.

ფენა-ფენა

1. Ingestion

ტექნოლოგია	დანიშნულება
PyMuPDF	PDF პარსინგი (ტექსტი, ცხრილები, სუ...
Tesseract (OCR)	სკანირებული PDF-ების ტექსტად გარდა...
python-docx	DOCX დოკუმენტების პარსინგი
unstructured	მრავალფორმატიანი ფაილების (PDF/DOC...
Gemini structured extraction	კანონიკური სქემა `Tender (CPV, შემ...

რატომ: საქართველოს სახელმწიფო შესყიდვების დოკუმენტაცია მრავალფორმატიანია — PDF-დან და DOCX-დან HTML-მდე. ერთიანი პარსერი გვაძლევს საშუალებას, ტენდერის ტექსტი ერთგვაროვანი სტრუქტურით მივიღოთ, ხოლო Gemini-ს structured extraction — კანონიკურ ველებად გარდაქმნას.

2. Embeddings

ტიპი	მოდელი	დანიშნულება
Dense	`gemini-embedding-001`	სემანტიკური მნიშვნელობ...
Sparse	`BAAI/bge-m3`	ლექსიკური წონები — ზუს...

რატომ ჰიბრიდული:

- Dense ვექტორი ვერ პოულობს "45233140-2" (გზების სამუშაოების CPV კოდი), თუ კორპუსში იგი იშვიათად გვხვდება;
- Sparse ვექტორი კარგავს სემანტიკურ კონტექსტს — "გზის რეაბილიტაცია" vs "გზის მშენებლობა" მსგავსია ლექსიკურად, მაგრამ სხვადასხვა ტენდერია;
- ერთად = ზუსტი recall + ზუსტი precision.

3. Vector Store — Qdrant (hybrid search)

მახასიათებელი	ღირებულება
---------------	------------

Named vectors	native dense+sparse ერთ წერტილში
Payload ფილტრები	CPV/ვადა/რეგიონი/ღირებულება/სერტიფ...
Fusion	RRF (Reciprocal Rank Fusion) ან DB...
Query API	dense+sparse prefetch fusion t...

რატომ Qdrant:

- native hybrid support — ცალ-ცალკე dense და sparse ვექტორების ერთ წერტილში შენახვის შესაძლებლობა;
- მდიდარი payload filtering — eligibility შეზღუდვები (ვადა, მინიმალური ფასი, რეგიონი) პირდაპირ ძებნისას;
- RRF/DBSF fusion — ორი retrieval შედეგის ერთ რანჟირებაში გაერთიანება;
- წონა-მსუბუქი self-hosted და managed რეჟიმებში.

4. Reranker — bge-reranker-v2-m3

პარამეტრი	მნიშვნელობა
ტიპი	cross-encoder
დანიშნულება	top-N (50–100) top-K (5–10)
ღირებულება	ძვირი ერთეულზე, მაგრამ იაფი top-N-...

რატომ cross-encoder:

- Bi-encoder (embedding-ზე დაფუძნებული) კარგია recall-ისთვის, მაგრამ ცუდია მცირე კანდიდატთა ჯგუფში გარჩევადობისთვის;
- Cross-encoder ერთდროულად ხედავს query-ს და document-ს უფრო ზუსტი შეფასება;
- გამოიყენება მხოლოდ top-N-ზე — ჯამური ღირებულება მინიმალურია.

5. Relational Store — PostgreSQL

რა ინახება	დანიშნულება
კომპანიის პროფილი	კომპეტენციები, სერტიფიკატები, გამო...
ისტორიული awards	წინა ტენდერების გამარჯვებული ფასები
Price model	$\text{win_prob} \times \text{margin}$ მაქსიმიზაცია
Audit trail	ყველა რეკომენდაციის ისტორია

რატომ PostgreSQL:

- ACID ტრანზაქციები ფასის მოდელისთვის;
- მდიდარი SQL ფუნქციები ანალიტიკისთვის;
- სანდო, მასშტაბირებადი, ფართოდ გავრცელებული.

6. LLM — Gemini (gemini-2.5-pro/flash)

რეჟიმი	მოდელი	გამოყენება
ღრმა ანალიზი	`gemini-2.5-pro`	ფასის რეკომენდაცია, el...
სწრაფი რეზიუმე	`gemini-2.5-flash`	ტენდერის შეჯამება, KPI...

რატომ Gemini:

- მრავალენოვანი (ქართული ენის ჩათვლით);
- ფართო context window — გრძელი ტენდერის დოკუმენტების ჩატანა ერთ pass-ში;
- structured output — JSON სქემის სტაბილური დაცვა;
- ხელმისაწვდომი API და გონივრული ფასი.

7. Frontend — React / Next.js Dashboard

კომპონენტი	ფუნქცია
KPI მონიტორინგი	win-rate, საშუალო მარჟა, პროგნოზირ...
ტენდერების სია	win-prob-ით, eligibility სტატუსით,...
დეტალური ხედი	ციტირებადი ინსაიტი, წყაროს ბმულები
ქართულენოვანი შეჯამება	ბუნებრივი ენა, არა ლიტერატურული თა...

ალტერნატივები, რომლებიც განვიხილეთ

ფუნა	ჩვენი არჩევანი	ალტერნატივა	რატომ ავირჩიეთ ჩ...
Dense embedding	Gemini	OpenAI `text-emb...	ქართული ენის უკე...
Sparse embedding	BGE-M3	SPLADE / BM25	ნასწავლი წონები ...
Vector store	Qdrant	Weaviate / Milvu...	native hybrid se...
Reranker	bge-reranker-v2-...	Cohere Rerank / ...	ღია მოდელი, ლოკა...
LLM	Gemini	Claude / OpenAI	ხარჯის/ხარისხის ...
Object storage	MinIO	S3 / GCS	self-hosted, ფასი

Pipeline — საფეხურების დეტალები

> ქვემოთ მოცემული დეტალები არ წარმოადგენს დოკუმენტის ფოკუსს — იკითხება დამატებითი დროის არსებობისას.

Pipeline შედგება ხუთი ძირითადი საფეხურისგან:

- 1.. ინჯესტი
- 2.. შენახვა
- 3.. ჰიბრიდული ძებნა
- 4.. რერანკინგი + win-prob სქორინგი
- 5.. ფასის რეკომენდაცია

1. ინჯესტი

მიზანი: სახელმწიფო შესყიდვების ტენდერების (ITT/RFP) დოკუმენტების ავტომატური შეგროვება და ნორმალიზება.

ნაბიჯები:

- 1.. წყაროდან წამოღება — სახელმწიფო შესყიდვების პორტალი / API;
 - 2.. პარსინგი — PyMuPDF (PDF), Tesseract (OCR-ით სკანირებული), python-docx (DOCX), unstructured (HTML);
 - 3.. სტრუქტურირებული ამოღება — Gemini structured extraction კანონიკური სქემა: ``
Tender { cpv: string, // CPV კოდი (მაგ: 45233140-2) buyer: string,
// შემსყიდველი deadline: datetime, // ვადა value: number, //
სავარაუდო ღირებულება requirements: [...], // მოთხოვნები demand_level: string
// მოთხოვნის დონე (მაღალი/საშუალო/დაბალი) } ``
 - 4.. Chunking — გრძელი დოკუმენტების სემანტიკური ნაწილებად დაყოფა (embedding-ისთვის);
 - 5.. Scheduler — ყოველდღიური ingestion ახალი ტენდერებისთვის.
- შედეგი: ტენდერის ნორმალიზებული ჩანაწერი + ჩანკები embedding-ისთვის.

2. შენახვა

მიზანი: მონაცემების ეფექტური შენახვა ჰიბრიდული ძებნისთვის.

Qdrant point-ის სტრუქტურა:

```
{  
  "id": "tender_45233140_2026_0042",  
  "vectors": {  
    "dense": [/* gemini-embedding-001 vector */],  
    "sparse": {/* BGE-M3 sparse representation */}  
  },  
  "payload": {  
    "cpv": "45233140-2",  
    "buyer": "თბილისის მერია",  
    "deadline": "2026-08-15T18:00:00Z",  
    "value": 1850000.0,  
    "region": "თბილისი",  
    "requirements": ["ISO 9001", "გამოცდილება ≥5 წელი"],  
    "demand_level": "მაღალი",  
    "raw_file_url": "minio://tenders/2026/0042.pdf"  
  }  
}
```

PostgreSQL-ში ინახება:

- კომპანიის პროფილი (კომპეტენციები, სერტიფიკატები);
- ისტორიული award-ები (წინა გამარჯვებული ფასები);
- price model-ის პარამეტრები.

Object Storage (MinIO/S3):

- raw PDF/DOCX ფაილები — ციტირებისთვის და აუდიტისთვის.

3. ჰიბრიდული ძებნა

მიზანი: ფართო retrieval — recall-ის მაქსიმიზაცია eligibility ფილტრებით.

ნაბიჯები:

- 1.. Query parsing — მომხმარებლის query სტრუქტურირებული ძებნის პარამეტრები;
 - 2.. Pre-filter — payload-ზე eligibility შეზღუდვები: - ვადა — მოქმედი ტენდერები; - ღირებულება — კომპანიის ბიუჯეტის ფარგლებში; - რეგიონი — გეოგრაფიული შეზღუდვა (საჭიროების მიხედვით); - სერტიფიკატები — მოთხოვნილი სერტიფიკატების მქონე;
 - 3.. Dense + Sparse Prefetch — Qdrant Query API: - dense vector search (gemini-embedding-001); - sparse vector search (BGE-M3); - ორივე პარალელურად;
 - 4.. Fusion — RRF (Reciprocal Rank Fusion) ან DBSF:
$$RRF_score(d) = \sum \frac{1}{k + rank_i(d)}$$
 სადაც rank_i(d) არის document-ის რანკი მე-i ძებნაში;
 - 5.. Top-K — შერწყმული შედეგიდან top-K (მაგ: 50–100) კანდიდატი.
- შედეგი: ფართო კანდიდატთა სია recall-ის მაქსიმუმით.

4. რერანკინგი + win-prob სკორინგი

მიზანი: precision-ის მაქსიმიზაცია მცირე კანდიდატთა ჯგუფზე.

4.1 Reranking (bge-reranker-v2-m3)

- შეყვანა: top-K კანდიდატი retrieval-დან;
- პროცესი: cross-encoder ერთდროულად აფასებს (query, document) წყვილებს;
- შედეგი: top-N (მაგ: 5–10) ყველაზე რელევანტური ტენდერი.

4.2 Win-probability სკორინგი

გადარჩევის შემდეგ ყოველ ტენდერს ენიჭება მოგების ალბათობა:

```
win_prob = f(
    ისტორიული_მოგება, // კომპანიის წინა win-rate ანალიზურ ტენდერებზე
    კონკურენცია,       // მოსალოდნელი მონაწილეო რაოდენობა
    მარფა               // პოტენციური მოგების მარფა
)
```

4.3 ახსმადლობა

ყველა შეფასებას თან ახლავს მიზეზი:

- რომელი მოთხოვნა დაემთხვა;
- რატომ მოხვდა მაღლა (ან დაბლა);
- რა რისკები არსებობს (მაგ: ვადა მცირე, კონკურენცია მაღალი).

5. ფასის რეკომენდაცია

მიზანი: რეკომენდებული ფასის შეფასება, რომელიც ამარჯვებს და მაქსიმალურ მარჯას იძლევა.

ფორმულა:

```
recommended_price = argmax(win_prob(price) × margin(price))
```

სადაც:

- `win_prob(price)` — ფასის ფუნქცია, რომელიც ისტორიულ მონაცემებზეა დაფუძნებული (რაც უფრო დაბალი ფასი, მით უფრო მაღალი `win_prob`);
- `margin(price)` — მარჯა ფასსა და თვითღირებულებას შორის.

ნაბიჯები:

- 1.. ისტორიული awards — PostgreSQL-დან წინა ანალოგიური ტენდერების გამარჯვებული ფასები;
- 2.. Price model — `win_prob × margin` მაქსიმიზაცია თითოეულ ფასის წერტილზე;
- 3.. Gemini-ით დასაბუთება — LLM ხსნის, რატომ ეს ფასია რეკომენდებული (ციტირებით);
- 4.. დიაპაზონი — მინიმალური, რეკომენდებული და მაქსიმალური ფასი (მოლაპარაკების სივრცე).

შედეგი: ციტირებადი ფასის რეკომენდაცია + ახსნა-განმარტება.

შეჯამება

საფეხური	იაფი/ძვირი	recall/precision	შედეგი
Retrieval (dense...	იაფი	recall	top-K კანდიდატი
Reranking (cross...	ძვირი	precision	top-N ზუსტი
Win-prob სქორინგი	იაფი	ბიზნეს-ლოგიკა	აღბათობა + ახსნა
Gemini (RAG)	ძვირი	ფინალური ინსაიტი	ციტირებადი რეკომ...

პრინციპი: ჯერ ფართო (იაფი), შემდეგ ზუსტი (ძვირი). ეს არის Atlas-ის არქიტექტურის გული.

პროექტის წესები (CLAUDE.md)

Project Identity

Atlas — B2B ტენდერების ინტელექტუალური პლატფორმა საქართველოს ბიზნესებისთვის.
ჰაკათონის პროექტი — საქართველოს ტექნიკური უნივერსიტეტი, 2026.
არ არის ძველი Exsul-ის გაგრძელება — ეს ახალი, დამოუკიდებელი პროექტია.

Product Vision

პლატფორმა ეხმარება კომპანიებს, რომლებიც სახელმწიფო შესყიდვების ტენდერებში (ITT/RFP) მონაწილეობენ:

- ანალიზებს კომპანიის კომპეტენციების პროფილს;
- ედრება მას ტენდერების კორპუსს;
- შეარჩევს მაღალი მოგების შანსის მქონე ტენდერებს;
- სთავაზობს რეკომენდებულ ფასს;
- უზრუნველყოფს ტექნიკური გულის ციტირებად ინსაიტს RAG-ით.

Core Architectural Principles

- 1.. ორფაზიანი retrieval: - ფართო (იაფი) recall-ის მაქსიმიზაცია; - ზუსტი (ძვირი) precision-ის მაქსიმიზაცია.
- 2.. ჰიბრიდული ძებნა: dense (სემანტიკა) + sparse (ლექსიკა/CPV);
- 3.. ციტირებადობა: ყველა რეკომენდაცია მოყვება წყაროს მითითებით;
- 4.. ახსნადობა: მოდელი არ უნდა იყოს "შავი ყუთი".

Tech Stack (high-level)

ფენა	ტექნოლოგია
Ingestion	PyMuPDF, Tesseract (OCR), python-d...
Dense Embeddings	`gemini-embedding-001`
Sparse Embeddings	`BAAI/bge-m3`
Vector Store	Qdrant (hybrid search, native dens...
Reranker	`bge-reranker-v2-m3` (cross-encode...
Relational	PostgreSQL (price model, audit tra...
LLM	Gemini (`gemini-2.5-pro/flash`)
Frontend	React / Next.js dashboard

Branch / Layer Model

ფენა	პასუხისმგებელი	დეტალი
`ingestion/`	დოკუმენტების პარსინგი	PDF/DOCX/HTML normal...
`embedding/`	dense + sparse ვექტორე...	gemini + BGE-M3
`storage/`	Qdrant + PostgreSQL + ...	persistence
`retrieval/`	ჰიბრიდული ძებნა	Query API + payload fi...
`rerank/`	cross-encoder + win-pr...	precision layer
`generation/`	Gemini RAG	insight + recommendati...
`frontend/`	React/Next.js dashboard	KPI + summaries

წესი: ფენა-ფენა იზოლაცია. UI არ ეხება Qdrant-ს პირდაპირ. LLM არ იძლევა პირდაპირ SQL-ზე წვდომას.

Georgian Language Requirements

გეორგიანული ენის ხარისხი — პროდუქტის გული.

- არ გამოიყენოთ ლიტერატურული თარგმნა;
- გამოიყენეთ მნიშვნელობაზე დაფუძნებული ლოკალიზაცია;
- შეინახეთ ბიზნეს-ტონი (ოფიციალური, მაგრამ არა ხელოვნური);
- მოერიდეთ რუსულ კალკებს;
- გამოიყენეთ ქართული ბიზნეს-ტერმინოლოგია;
- გადაამოწმეთ ქართული ტექსტი გამოტანამდე.

Performance & Quality Rules

- არ დაბლოკოთ UI: მძიმე ოპერაციები — async tasks-ში;
- არ გამოიყენოთ მთლიანი DB LLM-თან — მხოლოდ top-N რელევანტური კონტექსტი;
- Pagination: ყველა DB query გვერდებად იყოს დაყოფილი;
- მონიტორინგი: retrieval/rerank/gen latency-ის ლოგირება;
- CI: lint + type-check + test ყოველ commit-ზე.

Security Rules

- Never commit API keys. გამოიყენეთ environment variables ან secret manager;
- Never log API keys. ლოგებში — მხოლოდ status code და metadata;
- Never expose secrets to frontend. ყველა API call უნდა იყოს backend-ში;
- Validate AI JSON გამოყენებამდე;
- Treat AI output as untrusted — არანაირი SQL/shell execution AI output-იდან;
- Sanitize imported files — დოკუმენტების პარსინგი უსაფრთხო კონტექსტში.

Forbidden Behavior

- არ შექმნათ "შავი ყუთის" მოდელი — ყოველთვის მიუთითეთ წყარო;
- არ გამოიყენოთ მთლიანი DB LLM-თან;

- არ გამოიყენოთ ლიტერატურული თარგმნა ქართულ ენაზე;
- არ დაბლოკოთ UI thread;
- არ შეინახოთ raw API key-ები ფაილებში;
- არ გამოიყენოთ AI output SQL/shell command-ად.

Response Format

დავალების დასრულებისას მოახსენეთ:

- 1.. Files changed — რა ფაილები შეიცვალა;
- 2.. Stack impact — რომელ ფენაზეა გავლენა;
- 3.. Commands run — რა ბრძანებები გაეშვა;
- 4.. Test/Build result — წარმატებული იყო თუ არა;
- 5.. Known risks — რა რისკები რჩება;
- 6.. Next recommended step — რა უნდა გაკეთდეს შემდეგ.

პრაქტიკული იყავი. ააშენე რაც გამოგაქვს. არ მორთო ცხედარი პულსის შემოწმებამდე.

შინაარსი

მიმოხილვა	2
არქიტექტურა (ერთი შეხედვით)	2
ტექნოლოგიური სტეკი (მოკლედ)	2
დოკუმენტაციის რუკა	3
ლიცენზია	3
სისტემური მიმოხილვა	4
მონაცემთა ნაკადი (Data Pipeline)	4
ძირითადი კომპონენტები	5
1. Ingestion Layer	5
2. Embedding Layer	5
3. Storage Layer	5
4. Retrieval Layer	5
5. Rerank Layer	5
6. Generation Layer	5
7. Frontend	6
დიზაინის პრინციპები	6
ცენტრალური იდეა	7
ფენა-ფენა	7
1. Ingestion	7
2. Embeddings	7
3. Vector Store — Qdrant (hybrid search)	7
4. Reranker — bge-reranker-v2-m3	8
5. Relational Store — PostgreSQL	8
6. LLM — Gemini (gemini-2.5-pro/flash)	8
7. Frontend — React / Next.js Dashboard	9
ალტერნატივები, რომლებიც განვიხილეთ	9
1. ინჟესტი	10
2. შენახვა	10
3. ჰიბრიდული ძებნა	11
4. რერანკინგი + win-prob სკორინგი	11
4.1 Reranking (bge-reranker-v2-m3)	11
4.2 Win-probability სკორინგი	11
4.3 ახსმადობა	11
5. ფასის რეკომენდაცია	11
შეჯამება	12
Project Identity	13
Product Vision	13
Core Architectural Principles	13

Tech Stack (high-level)	13
Branch / Layer Model	14
Georgian Language Requirements	14
Performance & Quality Rules	14
Security Rules	14
Forbidden Behavior	14
Response Format	15